

2 Лабораторная работа «Перевод исходной программы в обратную польскую запись»

2.1 Понятие обратной польской записи

Обратная польская запись (ОПЗ) - представляет собой одну из форм записи выражений и операторов, отличительной особенностью которой является расположение аргументов (операндов) перед операцией (оператором).

Например, выражение, записанное в обычной скобочной записи,

$$(a + d) / c + b * (e + d) ,$$

в ОПЗ имеет следующее представление:

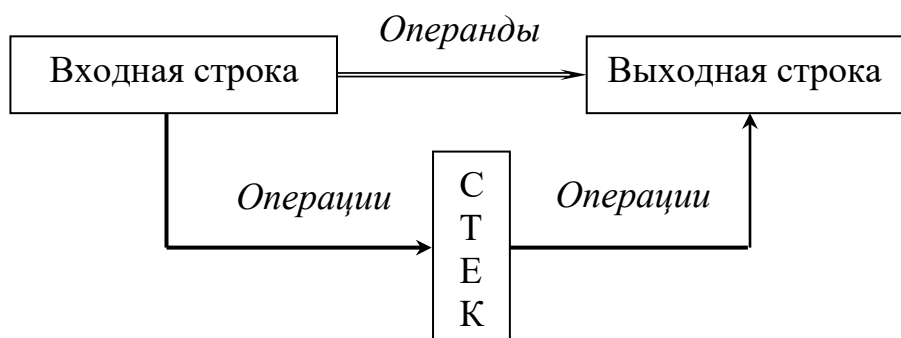
$$a d + c / b e d + * + .$$

Обратная польская запись получила широкое распространение благодаря своему основному преимуществу ОПЗ может быть вычислена за один просмотр цепочки слева направо, который часто называют проходом.

2.2 Алгоритм Дейкстры

Исследованию формальных способов преобразования арифметических и логических выражений в ОПЗ посвящены многочисленные исследования, однако в практике системного программирования наибольшее распространение получили способы преобразования на основе алгоритма Дейкстры.

Суть алгоритма Дейкстры можно представить следующим рисунком:



Из этого рисунка следует, что на вход алгоритма посимвольно поступает исходное выражение. Операнды исходного выражения пропускаются на выход и формируют так же посимвольно выходную строку. Операции обрабатываются по определенным правилам на основе стека.

Для реализации такой обработки известно в системном программировании понятие стека используется также в алгоритме Дейкстры для размещения в нем операций. При этом предварительно каждой операции приписывается свой приоритет на основе таблицы приоритетов, которая приведена ниже (табл. 2.1).

Таблица 2.1

Входной элемент	Приоритет
(	0
)	1
V	2
&	3
¬	4
Отношения	5
+	6
-	
*	7
/	
возведение в степень	8

С учетом этой таблицы сформулируем правила работы со стеком.

Если рассматриваемый символ является операцией, то с ним производятся следующие действия путем сравнения его приоритета с приоритетом верхнего символа стека:

- Если стек пуст, операция заносится в стек.

- Если в стеке верхний элемент (операция) имеет более высокий приоритет, то рассматриваемая операция проталкивается в стек.
- Если в стеке верхний элемент (операция) имеет более низкий или равный приоритет, то из стека в выходную строку выталкиваются все элементы (операции) до тех пор, пока не встретится операция с приоритетом выше, чем у рассматриваемой операции, после чего операция из входной строки проталкивается в стек.
- Если входной символ "(", то он всегда проталкивается в стек. Если входной символ ")", то он выталкивает из стека все символы операций до ближайшей "(". Сами скобки взаимно уничтожаются и в выходную строку не попадают.

Приведем пример перевода выражения в ОПЗ:

Выходная строка	a		b	+		5	-	<	2		c	-	1		q	+	=	&
С Т Е К		+		<	-		&			-		=		+				
					<					&		&		=				
														&				
Входная строка	a	+	b	<	-	5	&		2	-	c	=	1	+	q	v		

2.3 Перевод выражений, содержащих переменные с индексами, в ОПЗ

В языках программирования используются массивы и их элементы, то есть переменные с индексами. Сами массивы могут быть как одномерными, так и многомерными. Реально в памяти ЭВМ любые массивы размещаются только в виде одномерных массивов, поэтому при трансляции программ необходимо решать проблему отображения многомерного массива в одномерный и организации доступа к его элементам.

Для решения этой проблемы вводится операция *вычисления адреса элемента массива* (АЭМ).

Операция АЭМ имеет k -мерного массива имеет $k + 1$ операнд:

- имя массива;
- индексы массива,

и ОПЗ представляется следующим образом:

$\langle \text{операнд1} \rangle \langle \text{операнд2} \rangle \dots \langle \text{операнд } k+1 \rangle \text{ } k+1 \text{ АЭМ,}$

где: $\langle \text{операнд1} \rangle$ - имя массива, $\langle \text{операнд2} \rangle, \dots, \langle \text{операнд } k+1 \rangle$ - индексы массива, $k+1$ – счетчик операндов.

Например, ОПЗ выражения

$(a+b[i+20,j])*c+d$

будет иметь вид

$a \text{ } \underline{b} \text{ } \underline{i+20} \text{ } + \text{ } \underline{j} \text{ } \underline{3} \text{ АЭМ} + c * d +$

В этой ОПЗ операнды операции АЭМ подчеркнуты.

Для реализации алгоритма Дейкстры с учетом операции АЭМ, ее необходимо ввести в таблицу приоритетов, которая теперь выглядит следующим образом (табл. 2.2).

Таблица 2.2

Входной элемент	Приоритет
([АЭМ	0
,)]	1
V	2
&	3
¬	4
Отношения	5
+ -	6
* /	7
возведение в степень	8

Правила работы со стеком при переводе выражений с индексами в ОПЗ принимают следующий вид:

- обычные знаки операций обрабатываются так же, как и в предыдущем случае;
- с открытием массива (идентификатор и «[») в стек заносится операция АЭМ со значением счетчика операндов равным 2;
- знак «»,» выталкивает из стека все символы до ближайшего АЭМ и наращивает счетчик операндов на 1;
- закрывающая скобка «]» выталкивает из стека операцию АЭМ со значением счетчика операндов, но сама в выходную строку не попадает.

Перевод рассмотренного выше выражения в ОПЗ по алгоритму Дейкстры имеет следующий вид:

Выходная строка		a		b		i		20	+	j	3АЭМ	+		c	*	d	+
С Т Е К	(		+		2АЭМ		+		3АЭМ		+		*		+		
			(		+		2АЭМ		+		(						
						(	+		(								
							(										
Входная строка	(	a	+	b	[	i	+	20	,	j	]	)	*	c	+	d	■

Примечание. При трансляции исходной программы в объектный код на машинном языке адреса обращения к элементам массива должны быть правильно оформлены. Для этого необходимо вычислить так называемую функцию упорядочивания, которая устанавливает взаимно однозначное соответствие между многомерным массивом и отображающим его одномерным массивом. Но поскольку в программах может использоваться множество массивов, то нужно еще и выбрать соответствующую данному массиву функцию упорядочивания. Обычно вся эта информация, касающаяся каждого массива, собирается транслятором в определяющий вектор массива, а сами определяющие вектора размещаются во временную таблицу транслятора. Таким образом, обращение к соответствующему элементу многомерного массива связано с обработкой определяющих векторов и вычислением функции упорядочивания, что является семантической процедурой, скрывающимся за символом операции АЭМ.

2.4 Перевод в ОПЗ выражений, содержащих указатели функций

В арифметических и логических выражениях могут использоваться функции.

Введем операцию *функция*, которая так же, как и АЭМ, имеет k операндов и записывается в ОПЗ в виде:

<операнд1><операнд2> ... <операндк> к Ф,

где <операнд1> - имя функции, а <операнд2>,...,<операндк> - операнды функции.

Пример. Обратная польская запись выражения

$$y - f(x, z, y + 2)$$

имеет вид

$$y \ f \ x \ z \ y \ 2 \ + \ 4 \ \Phi -$$

Таблица приоритетов для алгоритма Дейкстры модифицируется следующим образом (табл. 2.3).

Таблица 2.3

Входной элемент	Приоритет
([АЭМ Ф	0
,)]	1
V	2
&	3
¬	4
Отношения	5
+ -	6
* /	7
Возведение в степень	8

При обработке оператора "Функция" выполняются следующие действия:

- по входному символу '(', следующему за идентификатором, в стек заносится оператор Ф со значением счетчика, равным 1;
- по входному символу ';' из стека выталкивается все до последнего оператора Ф и значение счетчика наращивается на 1;
- по входному символу ')' из стека выталкивается все до последнего оператора Ф, значение счетчика наращивается на 1 и оператор Ф выталкивается в выходную строку.

Например, для приведенного выражения процесс перевода в ОПЗ имеет вид:

Выходная строка	y		f		x		z		y		2	+	4Ф	-
С Т Е К		-		1Ф		2Ф		3Ф		+		4Ф	-	
				-		-		-		3Ф		-		
										-				
Входная строка	y	-	f	(	x	,	z	,	y	+	2	)		■□

Примечание. При трансляции исходной программы в объектный код на машинном языке функции должны быть оформлены как подпрограммы. Обращение к данным подпрограммам строится на основе обработки операции *функция* в ОПЗ. Поэтому операция *функция* является символом в ОПЗ, а за этим символом скрывается ряд семантических процедур, связанных с правильным обращением к соответствующим подпрограммам. Обычно поиск требуемой подпрограммы организовывается следующим образом: заводится таблица имен функций и в этой таблице указываются адреса, по которым располагаются эти подпрограммы. При распознавании оператора *функция* в ОПЗ из нее извлекается имя подпрограммы и ее операнды, а далее в объектном коде вставляются стандартные процедуры обращения к подпрограмме.

2.5 Перевод условных выражений в ОПЗ

Пусть имеется условие вида:

$$y = \begin{cases} a + b, & x \leq 0 \\ c + d, & x > 0 \end{cases}$$

Это условие в языках программирования записывается как условное выражение вида:

`y = if x ≤ 0 then a + b else c * d;`

В отличие от простых выражений, где порядок выполнения действий определяется старшинством операций или скобками и в процессе выполнения программы этот порядок не меняется, в условных выражениях он изменяется в зависимости от значения условия (в нашем примере $x \leq 0$).

Таким образом, понятие условного выражения должно быть также введено в ОПЗ представления исходной программы. Для этого вводятся так называемые динамические деревья.

Рассмотрим ряд понятий и особенностей для динамических деревьев:

- узлы и ветви деревьев могут помечаться метками;
- пустой узел - это такой узел, в котором отсутствует операция. Из пустого узла может исходить любое количество ветвей. Пустой узел вводится для объединения нескольких последовательных действий или разветвления некоторого вычислительного процесса;
- условный оператор перехода по значению 'ложь'. Оператор имеет следующее представление:

`<условие> <метка> УПЛ`

- оператор безусловного перехода:

`<метка> БП`

С учетом введенных понятий ОПЗ выражения

`y + ( I F a > b THEN x + y ELSE x + 2 )`

принимает следующий вид:

`y a b > M1 УПЛ x1 + M2 БП M1 : x2 + M2 : +`

Для обобщения алгоритма Дейкстры на условные выражения требуется модифицировать таблицу приоритетов, исходя из следующих соображений. Нетрудно заметить, что IF играет роль открывающей скобки, а THEN и ELSE - запятых. Теперь с учетом данного обстоятельства модифицированная таблица приоритетов примет следующий вид (табл. 2.4).

Таблица 2.4

Входной элемент	Приоритет
IF ([АЭМ Ф	0
,)] THEN ELSE	1
V	2
&	3
$\neg$	4
Отношения	5
+ -	6
* /	7
возведение в степень	8

Обработка символов операций будет дополнена правилами обработки ключевых слов условных выражений следующим образом:

- символ IF из входной строки заносится в стек;

- символ THEN выталкивает в выходную строку все операции из стека до ближайшего IF. В стеке к символу IF добавляется рабочая метка Mi и после этого в выходную строку записывается часть Mi УПЛ;
- ELSE выталкивает из стека все символы до ближайшего IF Mi. IF Mi превращается в IF Mi+1 и в выходную строку выталкивается часть Mi+1 БП Mi:
- Символ «)» указывает на конец условного выражения и выталкивает из стека все символы до ближайшего символа «(», при этом сам IF уничтожается, а в выходную строку помещается метка Mi+1: .

Например, для приведенного выражения процесс перевода в ОПЗ имеет вид:

Выходная строка	y				a		b	>	M1 УПЛ	x		1	+	M2 БП M1:	x		2	+	M2:	-
С Т Е К		-	(	IF		>			IF M1		+			IF M2		+				
			-	(	IF				(	IF M1				(	IF M2					
				-					-	(				-	(		-			
										-					-					
Входная строка	y	-	(	IF	a	>	b	THEN	x	+	1	ELSE	x	+	2	)				■

Примечание. Описанная выше обработка условных выражений связана с правильной организацией передачи управления внутри объектного кода, что и составляет внутреннюю семантику обработки условных выражений в ОПЗ.

2.6 Перевод оператора присваивания в ОПЗ

Формат оператора присваивания имеет вид:

<переменная> := <выражение>

а его ОПЗ представляется следующим образом:

`<переменная> <выражение> :=`

Для включения в ОПЗ оператора присваивания необходимо решить вопрос о месте этого оператора в таблице приоритетов. При выборе приоритета будем руководствоваться следующими соображениями:

- приоритет оператора присваивания должен быть выше, чем у любой арифметической или логической операции, чтобы знаки не могли вытолкнуть оператор присваивания из стека;
- приоритет должен быть меньше, чем у знака ';':

С учетом этого, таблица приоритетов операций принимает следующий вид (табл. 2.5).

Таблица 2.5

Входной элемент	Приоритет
IF ([АЭМ Ф	0
, ;)] THEN ELSE	1
:=	2
V	3
&	4
¬	5
Отношения	6
+ -	7
* /	8
возведение в степень	9

2.7 Перевод оператора безусловного перехода и меток в ОПЗ

Формат оператора безусловного перехода имеет вид:

GOTO <метка>

а его ОПЗ представляется следующим образом:

<метка> БП

При установлении приоритета оператора безусловного перехода руководствуются такими соображениями, как и в случае оператора присваивания. Поэтому приоритеты этих операторов одинаковы.

Для перевода в ОПЗ меток из исходной программы вводится *операция метки*, которую для простоты обозначают символом ':'. Эта операция имеет наименьший приоритет.

Таблица приоритетов с учетом оператора безусловного перехода и операции метки принимает следующий вид (табл. 2.6).

Таблица 2.6

Входной элемент	Приоритет
IF ([АЭМ Ф	0
, ;)] THEN ELSE	1
:= GOTO	2
V	3
&	4
¬	5
Отношения	6
+ -	7
* /	8
Возведение в степень	9
:	10

Рассмотрим перевод в ОПЗ фрагмента программы, содержащего выше рассмотренные операторы и операции. Фрагмент исходной программы имеет следующий вид:

R 1 := A + B ;

```

GOTO M1 ;
R2 := C * D - A ;
M1 :

```

Перевод данного фрагмента программы в ОПЗ по алгоритму Дейкстры представлен ниже:

Выходная строка	R1		A		B	+	:=		M1	БП	R2		C		D	*	A	-	:=	M1		:	
С Т Е К		:=		+		:=		GOTO					:=	*		-		:=			:		
				:=									:=		:=								
Входная строка	R1	:=	A	+	B		;	GOTO	M1		;	R2	:=	C	*	D	-	A		;	M1	:	■

2.8 Перевод условного оператора в ОПЗ

Существует два вида условного оператора: полный

```
IF условие THEN оператор1 ELSE оператор2 ;
```

и неполный

```
IF условие THEN оператор ;
```

По аналогии с рассмотренным выше условным выражением для представления условного оператора также используются динамические деревья. В результате обхода данного динамического дерева ОПЗ условного оператора принимают вид:

- для полного условного оператора

```
условие M1 УПЛ оператор1 M2 БП M1: оператор2 M2:
```

- для неполного оператора

```
условие M1 УПЛ оператор M1:
```

В общем случае перевод условного оператора в ОПЗ по алгоритму Дейкстры совпадает с переводом условного выражения. Однако следует заметить, что в разных языках программирования в условных операторах принято различное употребление ограничителей ';' в условном операторе. В

этом случае функции ';' как концевого маркера оператора являются в разных языках различными. Например, в языке программирования Паскаль ';' всегда закрывает условный оператор. В этом случае этот знак должен вытолкнуть из стека IF, расставить соответствующие метки, но при этом сам IF в выходную строку не попадает.

2.9 Перевод описаний переменных и процедурных блоков в ОПЗ

В компиляторах обработка описаний переменных необходима для:

- выявления типов данных;
- подготовки информации для распределения памяти;
- выделения в объектной программе места для размещения данных.

Очевидно, что распределение памяти в объектной программе всегда должно предшествовать ее генерации. Поскольку исходная программа может содержать вложенные процедуры, то область действия идентификаторов, описанных во вложенных процедурах, является локализованной в пределах процедуры. Компилятор должен учитывать это обстоятельство и располагать информацией о начале и конце процедуры.

Приведем пример обработки описаний данных на базе описанного выше подмножества языка PL/1.

В этом языке программирования имеется несколько типов данных:

DEC FLOAT – десятичное число с плавающей точкой;

DEC FIXED – десятичное число с фиксированной точкой;

BIN FIXED FIXED – двоичное число с фиксированной точкой;

CHAR – символьная строка.

Сопоставим этим конструкциям языка в ОПЗ *операцию типа*, формат которой имеет вид:

Список переменных k ТИП,

где *k* – количество описываемых переменных;

$$ТИП = \begin{cases} DFT \\ DFD \\ BFD \\ CHAR \end{cases}$$

Поясним операцию типа, для чего предположим, что в программе описано несколько переменных

DCL (A, B, C) DEC FLOAT

С учетом операции типа построенная ОПЗ этого описания примет следующий вид:

A B C 3 DFT

Рассмотрим далее следующие операции описания процедур:

- *начало процедуры* - имеет два операнда: номер процедуры, уровень процедуры

<номер><уровень> НП

- *конец процедуры* - не имеет операндов

КП

- *конец описания* - имеет два операнда: номер процедуры, уровень процедуры

<номер><уровень> КО

Операторы начала и конца процедуры и описания данных в исходной программе являются неисполняемыми. Им можно присвоить самый низкий приоритет в таблица приоритетов, после чего она примет следующий вид (табл. 2.7).

Таблица 2.7

Входной элемент	Приоритет
IF ([АЭМ Ф	0
, ;)] THEN ELSE	1
:= GOTO	2
V	3
&	4
¬	5
Отношения	6
+ -	7
* /	8
возведение в степень	9
: PROC END DCL	10

Сам алгоритм Дейкстры с учетом обработки описательных операторов модифицируется следующим образом:

- символ PROC из входной строки заносится в стек, при этом в стек с ним также заносятся номер и уровень процедуры;
- символа DCL из входной строки заносится в стек с номером и уровнем процедуры и начальным значением счетчика операндов, равным 1;
- символ описания типа помещается в вершину стека;
- символ ',' из входной строки в зависимости от верхнего элемента стека выполняет различные функции:
 - если в вершине стека находится оператор DCL, значение счетчика операндов наращивается на 1 (запятая между операндами);
 - если в вершине стека находится символ описания типа, он выталкивает его из стека, помещает в выходную строку операцию ТИП со значением счетчика из оператора DCL и счетчик операндов принимает начальное значение 1 (запятая между символами описания типа);
- символ END из входной строки всегда заносится в стек;
- символ ';' из входной строки в зависимости от верхнего элемента стека выполняет различные функции:
 - если в вершине стека находится оператор PROC, то рассматриваемый символ выталкивает его из стека, занося в выходную строку операцию начала процедуры с ее номером и уровнем;
 - если в вершине стека находится оператор DCL, точка с запятой выталкивает его из стека, записывая в выходную строку операцию конца описания с номером и уровнем процедуры;
 - если в вершине стека находится символ описания типа, то рассматриваемый символ обрабатывает его аналогично запятой, а затем применяется к следующему в стеке оператору DCL;

- если в вершине стека находится оператор END, точка с запятой выталкивает его из стека, записывая в выходную строку операцию конца процедуры;
- символ '.' выталкивает из стека оператор END, записывая в выходную строку операцию конца процедуры.

Например, алгоритм перевода программы

```
PROC F1;
DCL (A,B) DEC FIXED,
      D DEC FLOAT;
PROC F2;
      DCL (C,D) BIN FIXED;
END;
END.
```

выглядит следующим образом:

Выходная строка		F1	1 1 HP			A		B	
С Т Е К	PROC 1,1			DCL 1,1,1	(		(		DCL 1,1,2
					DCL 1,1,1		DCL 1,1,2		
Входная строка	PROC	F1	;	DCL	(	A	,	B	)

Выходная строка		2 DFD	D		DFT	1 1 KO		F2	2 2 НП
С Т Е К	DFD	DCL 1,1,1		DFT	DCL 1,1,1		PROC 2,2		
	DCL 1,1,2			DCL 1,1,1					
Входная строка	DEC FIXED	,	D	DEC FLOAT	;		PROC	F2	;

Выходная строка			C		D			2 BFD	2 2 KO
С Т Е К	DCL 2,2,1	(		(		DCL 2,2,2	BFD	DCL 2,2,1	
		DCL 2,2,1		DCL 2,2,2			DCL 2,2,2		
Входная строка	DCL	(	C	,	D	)	BIN FIXED	;	

Выходная строка		КП		КП
СТЕК	END		END	
Входная строка	END	;	END	.

Таким образом, ОПЗ рассмотренной программы имеет следующий вид:

F1 1 1 НП A B 2 DFD D DFT 1 1 KO F2 2 2 НП C D 2 BFD 2 2 KO КП КП

2.10 Комплексный пример перевода исходной программы в ОПЗ

Рассмотрим комплексный пример перевода исходной программы на языке, описанном в лабораторной работе № 1, в обратную польскую запись.

PROC MAIN;

DCL (A1, A2) DEC FIXED;

```
A1=378;
```

```
A2=.73;
```

```
PROC CALC;
```

```
    DCL (SUM, MULT) DEC FIXED;
```

```
    IF A1+A2<>3.2 THEN GOTO P;
```

```
    SUM= (A1+A2) *A2;
```

```
P:    MULT=A1*A2;
```

```
END;
```

```
END.
```

Прежде всего, составим таблицу приоритетов, включив в нее все операции и операторы, имеющиеся во входном языке (табл. 2.8). Для этого воспользуемся обозначениями операторов и операций, которые были введены в лабораторной работе № 1.

Таблица 2.8

Приоритет	Входной элемент	Обозначение
0	IF	W6
	(	R4
1	THEN	W8
	.	R6
	,	R2
	;	R3
	)	R5
2	GOTO	W5
	=	O5
3	<	O3
	>	O4
4	+	O1
5	*	O2
6	PROC	W7
	END	W3
	DCL	W2
	:	O6

Привычное представление процесса перевода исходной программы в ОПЗ на основе алгоритма Дейкстры имеет следующий вид:

Выходная строка		MAIN	1 1 НП			A1		A2	
С Т Е К	PROC 1,1			DCL 1,1,1	(		(		DCL 1,1,2
					DCL 1,1,1		DCL 1,1,2		
Входная строка	PROC	MAIN	;	DCL	(	A1	,	A2	)

Выходная строка		2 DFD	1 1 КО	A1		378	:=	A2		.73	:=		CALC
С Т Е К	DFD	DCL 1,1,1			:=			:=				PROC 2,2	
	DCL 1,1,2												
Входная строка	DEC FIXED	;		A1	:=	378	;	A2	:=	.73	;	PROC	CALC

Выходная строка	2 2 НП			SUM		MULT				2 BFD	2 2 КО
С Т Е К		DCL 2,2,1	(		(		DCL 2,2,2	BFD	DCL 2,2,1		
			DCL 2,2,1		DCL 2,2,2			DCL 2,2,2			
Входная строка	;	DCL	(	SUM	,	MULT	)	BIN FIXED	;		

Выходная строка		A1		A2	+	3.2	>	M1 УПЛ		P	БП	M1:	SUM		A2		A1	+	:=
С Т Е К	IF		+					IF M1	GOTO		IF M1			:=		+		:=	
			IF		IF				IF M1							:=			
Входная строка	IF	A1	+	A2	>	3.2	THEN	GOTO	P		;	SUM	:=	A2	+	A1		;	

Выходная строка	P	:	MULT		A1			A2		SUM	+	*	:=		КП		КП
С Т Е К				:=		*	(	+		*	:=		END		END		
						:=	*	(		:=							
							:=	*									
									:=								
Входная строка	P	:	MULT	=	A1	*	(	A2	+	SUM	)	;	END	;	END	.	

Полученная в результате перевода ОПЗ программы имеет вид:

MAIN 1 1 НП A1 A2 2 DFD 1 1 КО A1 378 := A2 .73 := CALC2 2 НП

SUM MULT 2 BFD 2 2 КО A1 A2 + 3.2 > M1 УПЛ P БП M1: SUM A2 A1 + :=

P : MULT A1 A2 SUM + * := КП КП

Для наглядности в рассмотренном примере текст программы представлен на исходном языке, но в реальном трансляторе входной текст является результатом работы лексического анализатора. В нем уже выделены и классифицированы лексемы, но в остальном работа протекает в соответствии с описанным выше примером.